SafeHer

Teammate Setup and SOS/UI Test Guide

Download the isolated SOS/UI branch, configure a debug build, install it on a physical Android phone, and record all 13 test cases.

Repository: https://github.com/Ghige/SAFEHER
Branch: sos-ui-teammate-share
Commit: c3bb710c8bf5dda90c0b5c2bb9ecfdd83a285679
Document date: 2026-10-10

NO SECRETS IN THIS DOCUMENT
DO NOT SEND A REAL EMERGENCY DURING TESTING

These are behaviors to test. They are not claimed as verified on a physical device.
Unit tests do not make this stage production-ready.
Do not commit .env, local.properties, google-services.json, or keystores.
If you already have a dirty local clone, use a separate clean clone.

Audience: Android QA / teammate assigned to verify SOS/UI on the current-app share branch.

Document type: Setup and test procedure only. This is not a production-readiness certificate.

Document date: 2026-10-17

Share instructions: docs/TEAM_SHARE.md

Companion PDF: docs/SafeHer_Teammate_Setup_and_SOS_Test_Guide.pdf

Do not treat unit tests as physical-device proof. The behaviors in this guide are what to test, not results already verified on a phone.

Document control

Table: Field / Value

Repository	https://github.com/Ghige/SAFEHER
Branch	team-share-current
Required commit	Latest commit on team-share-current (git log -1)
Short hash	Use git rev-parse --short HEAD
Android project root	safeher_android/
Debug application ID	com.safeher.app.debug
Release application ID	com.safeher.app
Do not commit	.env, local.properties, google-services.json, Firebase Admin JSON, keystores, API keys
Safety	Do not send a real emergency. Use a controlled test backend and designated test contacts only.

Status labels used in this guide:

Table: Label / Meaning

VERIFIED	Confirmed from files on this branch / this repository
NOT VERIFIED	Could not be confirmed from source; record what you observe
REQUIRES OWNER	The repository owner must provide a private file or a reachable test backend
LIKELY BLOCKED	This branch as published may prevent the step on a physical phone

Table of contents

1. Safety rules
2. GitHub repository and branch
3. Prerequisites
4. Local configuration (no secrets in Git)
5. Build and launch
6. Expected AI / SOS flow
7. Known branch limitations before you start
8. Physical-device test checklist (T01-T13)
9. Troubleshooting
10. Test reporting
11. Appendix -- verified project settings

1. Safety rules

- AI distress detection alone must never trigger SOS.
- I'm okay must not trigger SOS.
- The first confirmation timeout (30 seconds) must not trigger SOS.
- Cancel on the second confirmation must not trigger SOS.
- The first confirmation must not call the emergency-alert trigger API.
- Only the explicit Send Emergency Alert action may continue to the existing five-second countdown and SOS

trigger.
- Manual SOS must continue to work independently.
- An active SOS must not create another danger confirmation.
- Do not send real emergency alerts during testing.
- Do not copy, paste, or commit secrets, credentials, personal phone numbers, or real emergency-contact details into Git or into this report.
- If the app crashes, stop testing that path, capture Logcat, record the test ID, and report the exact steps. Do not make random code edits.

2. GitHub repository and branch

2.1 Use a clean clone

If you already have a SAFEHER folder with local changes, do not overwrite, reset, clean, or discard that work. Clone into a new directory.

2.2 Install Git (Windows PowerShell)

If git --version fails:

```
winget install --id Git.Git -e --source winget
```

Close and reopen PowerShell after install. Confirm:

```
git --version
```

Android Studio and the Android SDK are installed with the Android Studio installer (section 3). Do not guess an Android Studio version; the exact installer name is NOT VERIFIED.

2.3 Clone, checkout, and verify (Windows PowerShell)

```
git clone https://github.com/Ghige/SAFEHER.git
cd SAFEHER
git fetch origin
git switch team-share-current
git status
git log -1 --oneline
```

Confirm you are on the share branch:

```
git rev-parse --abbrev-ref HEAD
git log -1 --oneline
```

Expected branch name:

```
team-share-current
```

If the branch name does not match, stop and ask the repository owner. Do not test a different branch and call it this share.

If you already cloned the repo without local changes:

```
cd SAFEHER
git fetch origin
git switch team-share-current
git status
git log -1 --oneline
```

If that clone has unpublished local changes, leave it alone and use a second clean clone instead.

2.4 Open the Android project

Android Studio must open safeher_android, not the monorepo root.

```
# After you are in the SAFEHER clone:
explorer.exe .\safeher_android
```

Then in Android Studio: File > Open and select SAFEHER\safeher_android.

If the studio command is on PATH (NOT VERIFIED on every PC):

```
studio.bat ".\safeher_android"
```

3. Prerequisites

3.1 Required on the tester machine

Table: Item / Notes / Status

Windows 10/11 | This procedure uses PowerShell | Assumed
Git | Required to clone and verify the commit | VERIFIED needed
Android Studio | Exact channel/version NOT VERIFIED | REQUIRES MANUAL SETUP
JDK 17 toolchain | compileOptions and kotlinOptions target JVM 17. Android Studio JBR is acceptable if it can compile Java 17 | VERIFIED target 17
Android SDK Platform 35 | compileSdk = 35, targetSdk = 35 | VERIFIED
Android SDK Platform-Tools | Provides adb | VERIFIED needed
Gradle wrapper 9.3.0 | safeher_android/gradle/wrapper/gradle-wrapper.properties | VERIFIED
Android Gradle Plugin 8.7.3 | safeher_android/gradle/libs.versions.toml | VERIFIED
Kotlin 2.0.21 | same catalog | VERIFIED
Physical Android phone | minSdk = 26 (Android 8.0) or higher | VERIFIED minSdk
USB cable | Data-capable cable | REQUIRES MANUAL SETUP
USB debugging enabled | Developer options on the phone | REQUIRES MANUAL SETUP
OEM USB drivers | Required on some Windows PCs if adb devices is empty | NOT VERIFIED per OEM
google-services.json | Real Firebase client file for both com.safeher.app and com.safeher.app.debug | REQUIRES OWNER
local.properties | Must contain a valid sdk.dir. Gitignored | REQUIRES MANUAL SETUP

3.2 Required only for live SOS (T06-T09, and popup if backend fusion is required)

Popup appearance on this branch depends on a successful backend getConfidence call after on-device HIGH sustained risk (VERIFIED in AiMonitoringCoordinator.kt). Manual and AI-assisted SOS both call alertRepository.triggerAlert after the five-second countdown (VERIFIED in SOSViewModel.kt).

Table: Item / Notes / Status

Test backend | FastAPI app.main:app from safeher_backend/ | VERIFIED entrypoint
PostgreSQL | Required by backend .env.example DATABASE_URL | VERIFIED required for a useful backend
Reachable API URL from the phone | Set DEBUG_API_BASE_URL in gitignored local.properties to your laptop LAN URL | REQUIRES MANUAL SETUP
Designated test contacts only | Never use real emergency contacts | Safety rule
AI_AUTO_SOS_ENABLED=false | Backend .env.example | VERIFIED in template

3.3 What GitHub will never contain

These files are gitignored and must not be copied from GitHub or committed:

- safeher_android/local.properties
- safeher_android/app/google-services.json
- safeher_backend/.env
- firebase-credentials.json / Admin SDK JSON
- *.jks, *.keystore, keystore.properties
- Private API keys, tokens, and personal phone numbers

Templates you may copy (they contain placeholders only):

- safeher_android/local.properties.example

- safeher_android/app/google-services.json.example
- safeher_backend/.env.example

The .example Google Services file will not talk to Firebase. A real google-services.json must come from the authorized project owner.

4. Local configuration

4.1 local.properties

Android Studio usually writes sdk.dir on first sync. If you create it yourself, copy the example and set your SDK path:

```
cd safeher_android
copy local.properties.example local.properties
```

Example shape (do not treat the user path as yours):

```
sdk.dir=C\:\\Users\\YourUser\\AppData\\Local\\Android\\Sdk
MAPS_API_KEY=
```

MAPS_API_KEY may be left empty. The debug build still compiles (VERIFIED: Gradle defaults missing key to an empty string). Maps tiles may fail; SOS/UI overlay tests do not require maps.

For a physical phone, add your laptop LAN URL (trailing slash recommended):

```
DEBUG_API_BASE_URL=http://<YOUR_LAPTOP_LAN_IPV4>:8000/
```

This branch includes DebugApiBaseUrl.kt. Debug physical devices use that override. Debug emulators ignore it and keep http://10.0.2.2:8000/. After changing the URL, Rebuild / Run so BuildConfig regenerates.

4.2 google-services.json

Ask the owner for the real file. Place it at:

```
safeher_android\app\google-services.json
```

The example file shows two Android clients:

- com.safeher.app
- com.safeher.app.debug

The debug APK uses com.safeher.app.debug (VERIFIED: applicationId + applicationIdSuffix = ".debug"). The owner file must include the debug client or Gradle / Firebase setup can fail.

4.3 Backend (only when testing live SOS or the confidence popup)

From safeher_backend (owner/dev machine):

```
cd safeher_backend
copy .env.example .env
py -3.11 -m venv .venv
.\.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
alembic upgrade head
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Keep in .env (template values, not secrets):

- OTP_PROVIDER=mock
- OTP_ALLOW_DEV_SHORTCUT=true
- AI_AUTO_SOS_ENABLED=false

Mock OTP 123456 is accepted when the shortcut is enabled (VERIFIED in otp_service.py). Use a disposable test

account, not a real emergency identity.

Confirm the API locally:

```
# On the machine running uvicorn
curl http://127.0.0.1:8000/docs
```

Windows Firewall must allow inbound TCP 8000 if a phone will reach the laptop. Uvicorn must bind 0.0.0.0, not only 127.0.0.1.

5. Build and launch

5.1 Open and sync

1. Open safeher_android in Android Studio.
2. Select the debug variant (com.safeher.app.debug).
3. Wait for Gradle sync.
4. If sync asks for SDK 35, install Android SDK Platform 35 from SDK Manager.
5. If sync fails on Google Services, you are missing a real google-services.json.
6. If sync fails on JDK, set Gradle JDK to Android Studio JBR or another JDK 17+ that can compile Java 17.

5.2 Build the debug APK from PowerShell

```
cd safeher_android
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
$env:Path = "$env:JAVA_HOME\bin;" + $env:Path
.\gradlew.bat :app:assembleDebug
```

JAVA_HOME must point to your Android Studio JBR if the path differs. Confirm:

```
java -version
```

Successful task: :app:assembleDebug.

5.3 Connect the phone

On the phone:

1. Enable Developer options (tap Build number seven times).
2. Enable USB debugging.
3. Connect USB. Accept the RSA prompt.

On the PC:

```
adb devices
```

Expected: one device listed as device (not unauthorized or offline).

If the list is empty, install the phone vendor USB driver, try another cable/port, and re-check USB debugging.

5.4 Install and launch

Android Studio: select the phone, click Run, debug variant.

Or PowerShell from safeher_android:

```
.\gradlew.bat :app:installDebug
adb shell am start -n com.safeher.app.debug/com.safeher.app.MainActivity
```

The launcher activity is com.safeher.app.MainActivity inside debug package com.safeher.app.debug (VERIFIED from AndroidManifest.xml and applicationIdSuffix).

5.5 Sign in

Use a test account on the test backend. If the owner's mock OTP shortcut is enabled, the backend accepts 123456. If login fails, treat later live-SOS tests as BLOCKED and still try on-device AI steps that do not need an authenticated session only if the app allows them. Profile and SOS APIs require login (expected for this app).

5.6 Grant permissions for the features under test

When prompted, or from the AI / SOS screens:

Table: Permission / Why / Manifest source
Microphone (RECORD_AUDIO) | AI monitoring | VERIFIED
Notifications (POST_NOTIFICATIONS) | AI foreground-service notification on Android 13+ | VERIFIED
Location (fine/coarse) | SOS coordinates | VERIFIED
Background location | Live tracking after SOS; not required to see the overlay | VERIFIED present
Ignore battery optimizations | Recommended before T13 | VERIFIED permission exists

Also confirm:

- AI Monitoring toggle is on from the AI tab (AI Monitoring / Listening for distress signals).
- Notification shade shows SafeHer AI Monitoring / Listening for distress signals in the background. when monitoring is on (VERIFIED strings).

5.7 Start AI monitoring

1. Open the app and reach Home / Dashboard (dashboard route).
2. Tap the center bottom item AI Monitor.
3. Grant microphone if asked.
4. Turn AI Monitoring on.
5. Open View Status if present. On-device models should read Ready when yamnet_embedding.tflite and safeher_classifier_v2.tflite loaded (VERIFIED assets and status strings).

5.8 Collect Logcat if something fails

Filter to the debug package. Do not record or share raw audio, phone numbers, or tokens.

```
adb logcat -d --pid=$(adb shell pidof -s com.safeher.app.debug)
```

If pidof is unavailable:

```
adb logcat -d | Select-String -Pattern "com.safeher.app.debug|TfliteAudioInferenceEngine|AiD
etectionLogger|AiMonitoringCoordinator|AudioMonitorService|SOSViewModel|DashboardNavigation|
AndroidRuntime"
```

Useful text that is safe to share (scores and flags, not PCM):

- SafeHer V2 audio inference pipeline ready
- SafeHer V2 distress score:
- AI audio frame model=
- AI backend /confidence
- AI user confirmation emergency=
- DashboardNavigation: navigate to SOS_ACTIVE
- SOSViewModel: emit NavigateToActive

Do not enable or request raw PCM logging.

If the process dies immediately, capture the crash:

```
adb logcat -d *:E AndroidRuntime:E
```

Stop the failing test. Record Test ID + steps. Do not edit application code.

6. Expected AI / SOS flow

This is the intended flow on team-share-current. It is not a claim that a physical device already passed.

```
Audio capture (AudioMonitorService, 16 kHz mono)
    -> TFLite inference (yamnet_embedding.tflite -> safeher_classifier_v2.tflite)
    -> Hybrid risk / sustained distress (AudioRiskEvaluator)
    -> Backend confidence (only if risk is HIGH, sustained, not impulsive)
    -> Screen-level help confirmation (overlay on current screen)
    -> I need help
    -> Second confirmation: Send an emergency alert?
    -> Send Emergency Alert
    -> Existing 5-second countdown (CountdownScreen / SOSViewModel)
    -> Active SOS
    -> Successful cancel
    -> Home / Dashboard
```

6.1 Confirmation copy (VERIFIED strings)

First step (help):

- Title: We noticed signs that you may need help
- Body: You're not alone -- someone can help you.
- Countdown: Confirm within N seconds (30 seconds, VERIFIED HELP_TIMEOUT_SECONDS = 30)
- Footnote: SafeHer will not send SOS automatically.
- Buttons: I need help / I'm okay

Second step (send alert):

- Title: Send an emergency alert?
- Body: Would you like SafeHer to send an emergency alert to your trusted contact?
- Buttons: Send Emergency Alert / Cancel

6.2 Safety rules mapped to this branch

Table: Rule / Implementation on this branch (to test, not yet device-proven)

Detection alone never sends SOS | Overlay must appear; countdown must not start by itself
I'm okay = no SOS | Calls onEmergencyConfirmation(false) only
30s timeout = no SOS | Help-step timeout calls onEmergencyConfirmation(false)
Cancel second step = no SOS | onCancelSendAlert -> confirmation false
First step does not trigger SOS API | I need help only advances to step 2; no coordinator confirm-true
Only Send Emergency Alert continues | onSendEmergencyAlert -> onEmergencyConfirmation(true) -> NavigateToSosCountdown
Confirm does not call AI SOS APIs | Coordinator emits countdown navigation; it does not call triggerAudioSos from confirm
Manual SOS independent | Home / SOS-tab onSosButtonClick -> countdown
No confirm during active SOS | Coordinator returns early if activeAlert.isOngoing
Overlay is screen-level | AiConfirmPromptHost is hosted in DashboardNavigation, not only on the AI tab

6.3 How to trigger a controlled distress recording

Do not create a real emergency or harm anyone.

1. Use a pre-recorded distress / scream sample played from another device or a speaker at moderate volume.

2. Keep the test phone's microphone unobstructed.

3. Play continuously for at least 5 seconds. On-device gating requires rolling confidence at or above 75 and duration of at least 5 seconds, plus other acoustic signals, before the backend is queried (VERIFIED AiDetectionConfig and AudioRiskEvaluator).

4. Inference runs about every 3 seconds while the app is in the foreground (VERIFIED INFERENCE_INTERVAL_MS = 3000).

5. After a handled confirmation, a 60 second cooldown applies (VERIFIED cooldownSeconds = 60). Wait before

the next distress test.

6. Quiet-room silence should not produce the popup. If it does, record it as a defect; do not change thresholds yourself.

If the popup never appears, check Logcat for AI audio frame and AI backend /confidence. Missing backend action CONFIRM_30S or FIRE_IMMEDIATELY means the overlay cannot start. Mark T01 BLOCKED rather than FAIL if the backend or URL is unreachable.

7. Known branch limitations (read before T01)

These are facts about team-share-current, not tester error.

1. Physical-phone API URL. Debug emulators use http://10.0.2.2:8000/. A physical phone must set DEBUG_API_BASE_URL in local.properties to the laptop LAN URL, then Rebuild. If that URL is missing or stale, register/login fail with ConnectException -- mark BLOCKED, not FAIL.

2. Popup needs backend fusion. After local HIGH sustained risk, the app calls aiRepository.getConfidence. Failure stores an error and does not show the overlay.

3. Background inference pauses. The inference loop runs only while _isAppInForeground is true. AI Status copy includes Background -- inference paused. T13 must record whether windows/inference actually continue; a visible FGS notification is not enough.

4. Movement AI is off. AiFeatureFlags.MOVEMENT_AI_ENABLED = false. Do not treat missing movement SOS as an SOS/UI failure.

5. Android AI_AUTO_SOS_ENABLED is compiled false. Backend template AI_AUTO_SOS_ENABLED=false. The confirm path must still not auto-send SOS.

6. SOS button hint vs code. UI text says Tap or hold / Hold for 3 seconds. The current SOSButton click path uses tap (detectTapGestures(onTap)). Use tap for T08/T09 unless you observe a working hold gesture.

7. Cancel returns to Dashboard. Active-alert back/cancel navigation targets route dashboard (VERIFIED). Testers should land on Home, not a stuck loader.

8. This guide does not change application code. If a limitation blocks a test, mark BLOCKED and describe what the owner must provide.

8. Physical-device test checklist

Print this section or copy it into your report. Use one status per test:

PASS / FAIL / BLOCKED / NOT RUN

Common preconditions for T01-T07 and T10-T13:

- Clean clone on team-share-current
- Debug app com.safeher.app.debug installed
- Signed in with a test account
- Microphone granted
- AI Monitoring on
- App in foreground unless the test says otherwise
- No active SOS unless the test starts from one
- Controlled recording only; no real emergency
- Wait 60 seconds after a previous confirmation/cooldown

T01 -- Danger popup (screen-level overlay)

Purpose: Confirm the help popup appears over the current screen, not only inside the AI Detection menu.

Preconditions: Common AI preconditions. Backend reachable enough to return CONFIRM_30S or

FIRE_IMMEDIATELY after sustained HIGH risk. If the phone cannot reach any API, mark BLOCKED.

Steps:

1. Start AI Monitoring from the AI tab.
2. Leave AI Detection: open Home (bottom Home).
3. Keep the app in the foreground on Home.
4. Play the approved controlled distress recording for at least 5-10 seconds.
5. Watch the current screen (Home), not only the AI tab.
6. If needed, repeat once after the 60-second cooldown. Do not raise volume to harmful levels.

Expected result:

- A dialog appears over Home with the first-step help copy.
- Footer shows SafeHer will not send SOS automatically.
- A seconds remaining label counts down from 30.
- No five-second SOS countdown and no Active Alert screen.
- Returning to the AI tab is not required to see the popup.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T02 -- I'm okay

Purpose: Dismissing the first step must close the popup and must not start countdown or SOS.

Preconditions: T01 popup visible on the help step. No active SOS.

Steps:

1. Trigger the help popup as in T01 (any foreground screen).
2. Tap I'm okay.
3. Wait at least 10 seconds.
4. Check Home, SOS tab, and Active Alert route.

Expected result:

- Popup closes immediately.
- No Sending SOS in? countdown.
- No Active Alert.
- Logcat may show AI user confirmation emergency=false.
- No triggerAlert / active-alert navigation.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T03 -- First timeout

Purpose: Ignoring the first popup for 30 seconds must not start SOS.

Preconditions: Help popup visible. Do not touch the dialog. No active SOS.

Steps:

1. Trigger the help popup as in T01.

2. Do not tap I need help or I'm okay.
3. Watch the Confirm within N seconds label reach 0.
4. Wait 10 more seconds after the dialog disappears.

Expected result:

- Popup closes by itself at 30 seconds.
- No countdown and no Active Alert.
- Confirmation logged as not an emergency if Logcat is available.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T04 -- Second confirmation

Purpose: I need help must open the send-alert step and must not start SOS yet.

Preconditions: Help popup visible. No active SOS.

Steps:

1. Trigger the help popup.
2. Tap I need help.
3. Read the new title and buttons.
4. Do not tap Send Emergency Alert.
5. Confirm you are still not on the countdown screen.

Expected result:

- Second dialog: Send an emergency alert?
- Buttons: Send Emergency Alert and Cancel.
- No 5-second SOS countdown.
- No Active Alert.
- Logcat must not show AI user confirmation emergency=true yet.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T05 -- Cancel second confirmation

Purpose: Cancel on the second step must not start countdown or SOS.

Preconditions: Second confirmation visible from T04. No active SOS.

Steps:

1. From the send-alert dialog, tap Cancel.
2. Wait 10 seconds.
3. Check Home, SOS tab, and that no countdown appears.

Expected result:

- Dialog closes.
- No countdown, no Active Alert, no trigger API success.
- Logcat may show AI user confirmation emergency=false.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T06 -- AI-assisted SOS (controlled test backend only)

Purpose: The full explicit path reaches the existing 5-second countdown and then Active SOS.

Preconditions: Test backend and designated test contacts only. Phone can actually reach that backend. If a physical phone still calls http://10.0.2.2:8000/, DEBUG_API_BASE_URL was not rebuilt into the APK -- mark BLOCKED. Do not use production WhatsApp recipients.

Steps:

1. Confirm you are on a test environment (ask the owner if unsure).
2. Repeat T01 and T04.
3. Tap Send Emergency Alert.
4. Observe the countdown screen title Sending SOS in? for 5 seconds.
5. You may tap Cancel SOS only if you must abort a mistaken real-looking send; otherwise let it finish for this test.
6. Confirm the Active Alert screen (Alert Active).

Expected result:

- Countdown is exactly the existing 5-second SOS countdown, not another 30-second help timer.
- After countdown, Active Alert is shown.
- Contacts notified must be test contacts only. If real contacts would be notified, abort and mark BLOCKED.
- This is the first time SOS trigger is allowed in the AI path.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T07 -- Cancel AI SOS

Purpose: Cancel the active AI-assisted alert, clear the session, and return to Home/Dashboard without an infinite loader.

Preconditions: T06 produced an Active Alert. Network online. If offline, the Cancel button should stay enabled and show You are offline. Connect to the internet to cancel this alert. -- record that, then reconnect before scoring T07.

Steps:

1. On Active Alert, confirm Cancel SOS Alert is enabled (even if a banner says offline).
2. If online, tap Cancel SOS Alert.
3. Confirm the dialog Cancel Emergency Alert? / Yes, I'm Safe.
4. Wait for navigation.
5. Confirm you are on Home (Home / Dashboard), not a perpetual Loading? screen.
6. Open the SOS tab and confirm it does not still show an ongoing active alert.

Expected result:

- Backend cancel succeeds on the test backend.
- Active session is cleared.
- UI shows Alert cancelled. Returning home? if the alert is no longer ongoing, then Home.
- No infinite loading.

- If cancel fails because of a missing alert id, you may see No active alert to cancel. -- record as FAIL if an alert was visibly active.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T08 -- Manual SOS from Home

Purpose: Home SOS still uses the same countdown and Active SOS, then cancel returns to Home.

Preconditions: No active SOS. Test backend if you will let countdown finish. Test contacts only.

Steps:

1. Open Home / Dashboard.
2. Tap the large SOS control (HomeSosSection / SOSButton). If hold does nothing, tap -- that is the implemented gesture on this commit.
3. Confirm countdown Sending SOS in?.
4. Let it finish only on the test backend.
5. Confirm Active Alert.
6. Cancel with Yes, I'm Safe.
7. Confirm return to Home/Dashboard.

Expected result:

- Manual path does not show the AI help overlay.
- Countdown is 5 seconds.
- Cancel returns to Home, not an infinite loader.
- If the trigger API cannot be reached, countdown may still appear; Active Alert will fail. Record BLOCKED (URL/backend) vs FAIL (UI bug) carefully.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T09 -- SOS tab flow and cancellation

Purpose: Exercise the Alerts / SOS tab (nav_alerts / route sos) and record any navigation or session differences vs Home.

Preconditions: No active SOS. Same test backend rules as T08.

Steps:

1. Tap bottom Alerts.
2. Read Emergency SOS / Press and hold the button below... (string) and the on-screen SOS button.
3. Tap the SOS button to start countdown.
4. Confirm countdown, then Active Alert (test backend only).
5. Cancel and record the screen you land on.
6. Note differences vs T08: start route, cancel landing screen, session restoration, banners.

Expected result:

- Same 5-second countdown and Active Alert behavior as Home.

- Cancel should return to Home/Dashboard (dashboard), not get stuck on sos/active.
- Record any extra Alerts-tab banners (preflight, offline, WhatsApp warning).

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T10 -- Duplicate prevention

Purpose: While SOS is active, controlled distress must not show another danger confirmation.

Preconditions: An Active Alert is on screen (from T06, T08, or T09). AI Monitoring still on. App in foreground.

Steps:

1. Keep the Active Alert screen visible.
2. Play the controlled distress recording for 10+ seconds.
3. Watch for any help / send-alert dialog.
4. Check Logcat for a new ShowEmergencyConfirmation / help-title dialog.

Expected result:

- No second danger confirmation.
- Active SOS remains the only emergency UI.
- Coordinator is specified to return early when activeAlert.isOngoing is true.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T11 -- AI regression (capture, windows, TFLite, sustained distress)

Purpose: Confirm the audio pipeline still runs. Do not share private audio.

Preconditions: AI Monitoring on. App in foreground. Models Ready on AI Status.

Steps:

1. Open AI Status if available and note model state and Foreground -- inference active.
2. In a quiet room, start Logcat capture.
3. Speak normally for 10 seconds, then play the controlled distress sample for 10 seconds.
4. Confirm the AI tab waveform / confidence / rolling confidence updates (if shown).
5. Inspect Logcat for the lines below. Never attach a microphone recording to the report.

Expected Logcat (safe):

- SafeHer V2 audio inference pipeline ready (at process start)
- Loaded YAMNet embedding model: yamnet_embedding.tflite
- Loaded SafeHer V2 classifier: safeher_classifier_v2.tflite
- Repeating SafeHer V2 distress score:
- Repeating AI audio frame model= with raw=, rolling=, risk=, dur=
- Optional: AI backend /confidence ... action=... when shouldQueryBackend is true

Expected result:

- Models load.
- Frames continue while foregrounded.

- Distress sample raises score / duration / risk vs quiet room (NOT VERIFIED numerically on your phone; record observed values).
- No raw PCM dumps in the log.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T12 -- No automatic SOS

Purpose: Distress detection alone must not start countdown or SOS. Only Send Emergency Alert may continue.

Preconditions: No active SOS. AI Monitoring on.

Steps:

1. Trigger distress until the help popup appears.
2. Do nothing for 5 seconds. Confirm no SOS countdown.
3. Tap I need help. Confirm still no SOS countdown.
4. Tap Cancel (or I'm okay on a fresh help popup after cooldown).
5. Repeat once more and tap Send Emergency Alert only if T06 is authorized; otherwise stop after step 4.

Expected result:

- Detection, help step, and I need help never start the 5-second SOS countdown.
- Only Send Emergency Alert starts that countdown.
- Footnote remains SafeHer will not send SOS automatically.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

T13 -- Background stability (only after foreground tests pass)

Purpose: Check monitoring with the app backgrounded and the screen locked. Verify actual audio-window creation and inference, not only that the notification remains visible.

Preconditions: T01 and T11 passed in the foreground. Microphone permission is still granted. Note battery-optimization state. Do this last.

Steps:

1. Start AI Monitoring. Confirm the SafeHer AI Monitoring notification.
2. Record: Android version, OEM, battery restriction (Unrestricted / Optimized / Restricted), and whether microphone access is "Allow all the time" vs "Only while using the app".
3. Start Logcat.
4. Press Home to background the app. Wait 30 seconds.
5. Lock the screen. Wait 30 seconds.
6. Play the controlled distress sample while locked.
7. Unlock and bring the app to the foreground.
8. Compare Logcat timestamps: did SafeHer V2 distress score / AI audio frame continue while backgrounded/locked, or did they stop?
9. Open AI Status: does it show Background -- inference paused when returning from background?

Expected result (what to record, not a guaranteed pass):

On this branch the coordinator stops the inference loop when the app is not in the foreground. A likely observation is:

- Notification may remain.
- AI audio frame / distress-score lines stop while backgrounded.
- Status can show Background -- inference paused.

If that is what you see, record it accurately. Score:

- PASS only if you observed continued windowing and inference while backgrounded/locked (unexpected on this commit, but possible if OEM behavior differs).
- FAIL only if the foreground-service notification itself vanishes immediately, the process is killed, or monitoring cannot be restarted, and that is not explained by OEM battery settings.
- BLOCKED if OEM settings prevent a fair test.
- Do not mark FAIL solely because inference paused -- that matches AiMonitoringCoordinator.startInferenceLoop on this commit. Describe it under actual result.

Status: PASS / FAIL / BLOCKED / NOT RUN

Actual result:

Screenshot or Logcat evidence:

Tester name / date:

9. Troubleshooting

Do not recommend or perform random code edits. If a crash occurs: stop, capture Logcat, record the Test ID, and write the exact steps.

Gradle sync failure

- Confirm you opened safeher_android, not the repo root.
- Confirm JDK can compile Java 17.
- Confirm internet access for first-time Gradle 9.3.0 / dependency download.
- Confirm a real app/google-services.json exists.

Missing SDK or JDK

- SDK Manager: Android SDK Platform 35, Platform-Tools.
- Settings > Build, Execution, Deployment > Build Tools > Gradle > Gradle JDK = Android Studio JBR.

Missing google-services.json

- Symptom: Google Services plugin error during sync/build.
- Fix: obtain the real file from the owner. Do not invent a JSON. Do not commit it.

USB debugging / ADB device not detected

```
adb kill-server
adb start-server
adb devices
```

Accept the phone RSA prompt. Try another cable. Install the OEM USB driver.

App crashes on launch

```
adb logcat -d AndroidRuntime:E *:F
```

Common owner-side causes: bad/missing google-services.json, Firebase client missing com.safeher.app.debug. Stop testing. Report stack + Test ID (launch).

Missing screen-level popup

Check in order:

1. AI Monitoring on, microphone granted, app in foreground.
2. Models Ready.
3. Distress played >= 5 seconds, not a door-slam / impulsive noise.
4. 60-second cooldown not active.
5. Logcat AI audio frame updating.
6. Logcat AI backend /confidence success with CONFIRM_30S or FIRE_IMMEDIATELY.
7. If backend fails, or a physical phone still uses 10.0.2.2 / a stale LAN IP, mark BLOCKED.

AI inference not running

- Look for SafeHer V2 audio inference pipeline unavailable or missing assets.
- Confirm RECORD_AUDIO.
- Confirm monitoring toggle on and app foregrounded.
- Confirm notification safeher_ai_monitoring.

Countdown appearing unexpectedly

- FAIL if countdown starts from detection, I'm okay, timeout, or second-step Cancel.
- Record the last button tapped and Logcat AI user confirmation / NavigateToSosCountdown.

SOS cancellation failure

- If offline: button should remain usable; message You are offline. Connect to the internet to cancel this alert.
- If No active alert to cancel.: note whether the UI still showed an active alert (FAIL) or the session was already cleared (record observation).
- Infinite Loading?: FAIL for T07.

Backend unreachable

- Emulator can use http://10.0.2.2:8000/ (LAN override is ignored on emulators).
- Physical phone needs DEBUG_API_BASE_URL=http://<laptop-lan-ip>:8000/ and a Rebuild. If Logcat still shows 10.0.2.2 or a stale IP, mark live-SOS tests BLOCKED.

Offline state incorrectly disabling functionality

- Active Alert cancel must stay enabled while offline (T07).
- Offline should not look like a dead button. If Cancel is disabled only because you are offline, FAIL.

App stopped by battery / background restrictions

- OEM "sleeping apps", "autostart", or Restricted battery can kill AudioMonitorService.
- Record the setting. Prefer Unrestricted for T13. This is an OS issue unless the FGS is never started in the foreground either.

10. Test reporting

Complete this summary after the 13 tests. Do not write READY FOR PRODUCTION. Unit tests and this checklist are not a production gate.

Environment

Table: Field / Value
Device model |
Android version |

App package | com.safeher.app.debug
App versionName | 2.0.0-debug (VERIFIED versionName + debug suffix)
versionCode | 1 (VERIFIED)
Git branch | team-share-current
Git commit |
Backend environment | test / none / unknown (never production)
Tester name |
Date |

Results

Table: Test ID / Name / Status (PASS / FAIL / BLOCKED / NOT RUN)
T01 | Danger popup |
T02 | I'm okay |
T03 | First timeout |
T04 | Second confirmation |
T05 | Cancel second confirmation |
T06 | AI-assisted SOS |
T07 | Cancel AI SOS |
T08 | Manual SOS |
T09 | SOS tab |
T10 | Duplicate prevention |
T11 | AI regression |
T12 | No automatic SOS |
T13 | Background stability |

Table: Count / Number
Passed |
Failed |
Blocked |
Not run |

Crashes

Table: Field / Value
Did any crash occur? | YES / NO
Test ID |
Exact steps |
Logcat file name |

Outstanding defects

- Defect 1:
- Defect 2:

Overall recommendation

Select exactly one:

- READY FOR FURTHER TESTING
- NOT READY

Do not select production-ready. This stage cannot declare the app production-ready from unit tests or from an incomplete device pass.

11. Appendix -- verified project settings

Inspected from team-share-current unless noted.

Table: Setting / Value / Source

namespace / applicationId | com.safeher.app | app/build.gradle.kts
debug suffix | .debug | same
debug package | com.safeher.app.debug | derived
minSdk | 26 | same
compileSdk / targetSdk | 35 | same
versionName / versionCode | 2.0.0 / 1 | same
debug versionName | 2.0.0-debug | versionNameSuffix
JVM / Kotlin jvmTarget | 17 | same
AGP | 8.7.3 | libs.versions.toml
Kotlin | 2.0.21 | same
Gradle wrapper | 9.3.0 | gradle-wrapper.properties
debug emulator URL | http://10.0.2.2:8000/ | DebugApiBaseUrl.kt
debug physical URL | DEBUG_API_BASE_URL in local.properties | DebugApiBaseUrl.kt
release BASE_URL | https://api.safeher.app/ | same
debug cleartext | permitted | app/src/debug/res/xml/network_security_config.xml
release/main cleartext | not permitted | app/src/main/res/xml/network_security_config.xml
Launcher activity | .MainActivity | AndroidManifest.xml
Audio FGS | AudioMonitorService / microphone | manifest + service
Location FGS | LocationForegroundService | manifest
Audio models | yamnet_embedding.tflite, safeher_classifier_v2.tflite | AiModelAssets.kt
Help timeout | 30 s | AiConfirmPromptViewModel
SOS countdown | 5 s | SOSViewModel.COUNTDOWN_SECONDS
Inference interval | 3000 ms | AiMonitoringCoordinator
On-device query gate | HIGH + sustained + not impulsive | AudioRiskEvaluator
Confidence / duration gate | 75.0 / 5.0 s | AiDetectionConfig
Detection cooldown | 60 s | AiDetectionConfig
Movement AI | false | AiFeatureFlags
Backend auto SOS template | AI_AUTO_SOS_ENABLED=false | .env.example
Mock OTP | 123456 when shortcut enabled | otp_service.py
Bottom nav Home | dashboard | DashboardRoutes
Bottom nav Alerts / AI / History / Profile | sos / ai / sos/history / profile | HomeBottomNavigation
Cancel destination | dashboard | DashboardNavigation
Timber debug tree | planted in debug | SafeHerApp.kt

Unverified from source (do not invent)

- Exact Android Studio installer name and version
- OEM USB driver package names
- Whether a given physical phone can reach any backend with this debug URL
- Whether Firebase / WhatsApp on the owner's test project will message only test contacts
- Physical-device pass/fail of T01-T13
- Whether hold-for-3-seconds works (code path observed is tap)
- Clean-machine install of this guide (not executed on a blank PC)

Owner must provide

1. Real google-services.json including com.safeher.app.debug.
2. A reachable test backend on the same Wi-Fi, plus the tester's own DEBUG_API_BASE_URL.
3. Confirmation that emergency contacts on the test account are designated test recipients only.
4. Optional: Firebase Admin JSON if they want backend FCM during the session.
5. Access to the private GitHub repository if the clone is denied.

End of guide

Source of truth for regeneration: this Markdown file.

PDF generator: scripts/generate_teammate_sos_test_guide_pdf.py.

No secrets are included in this document.